

Custom Compiler™ Analog Design Migration User Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	4
Related Products, Publications, and Trademarks	4
Conventions	5
Customer Support	6
Statement on Inclusivity and Diversity	6

1. Schematic Migration	8
The Schematic Migration Flow	10
Configuring the Setup Tab	11
Specifying Mapping Classes	14
Specifying Information for Cell Mapping	20
Specifying Information for Connecting Terminals	25
Saving the Mapping file and Starting Migration	27
Preserving Information After Migration	27

2. Layout Migration	28
Reviewing the Source Design	30
Preparing the Layout Parameters Configuration file	32
Extracting the Source Design Using Design Migration	33
Technology Map File Review and Import	37
Generating Target Design - Setup	40
Generating Target Design- Pin Placement	44
Generating Target Design- Routing	47
Generating Target Design– Pin Generation	49
Comparing the Migrated Design with Source Design	50

About This User Guide

This user guide explains how to use the Custom Compiler Schematic Editor to create electronic schematics.

This preface includes the following sections:

- [New in This Release](#)
- [Related Products, Publications, and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)
- [Statement on Inclusivity and Diversity](#)

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the Custom Compiler Release Notes on the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about the Custom Compiler tool, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following related Synopsys products:

- Synopsys® Custom Compiler™
- PrimeSim™ Continuum
- PrimeSim™
- PrimeSim™ Pro
- PrimeSim™ SPICE
- PrimeSim™ HSPICE®
- PrimeSim™ XA

- PrimeSim™ CCK
- PrimeSim™ MOSRA
- VCS® PrimeSim™ AMS
- Synopsys FineSim®
- Synopsys HSPICE®
- Synopsys VCS®
- Synopsys Verdi®
- Synopsys OptoCompiler™
- Synopsys StarRC™
- Mentor Graphics® Calibre®

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
Courier	Indicates syntax, such as <code>write_file</code> .
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.

Convention	Description
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working

environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

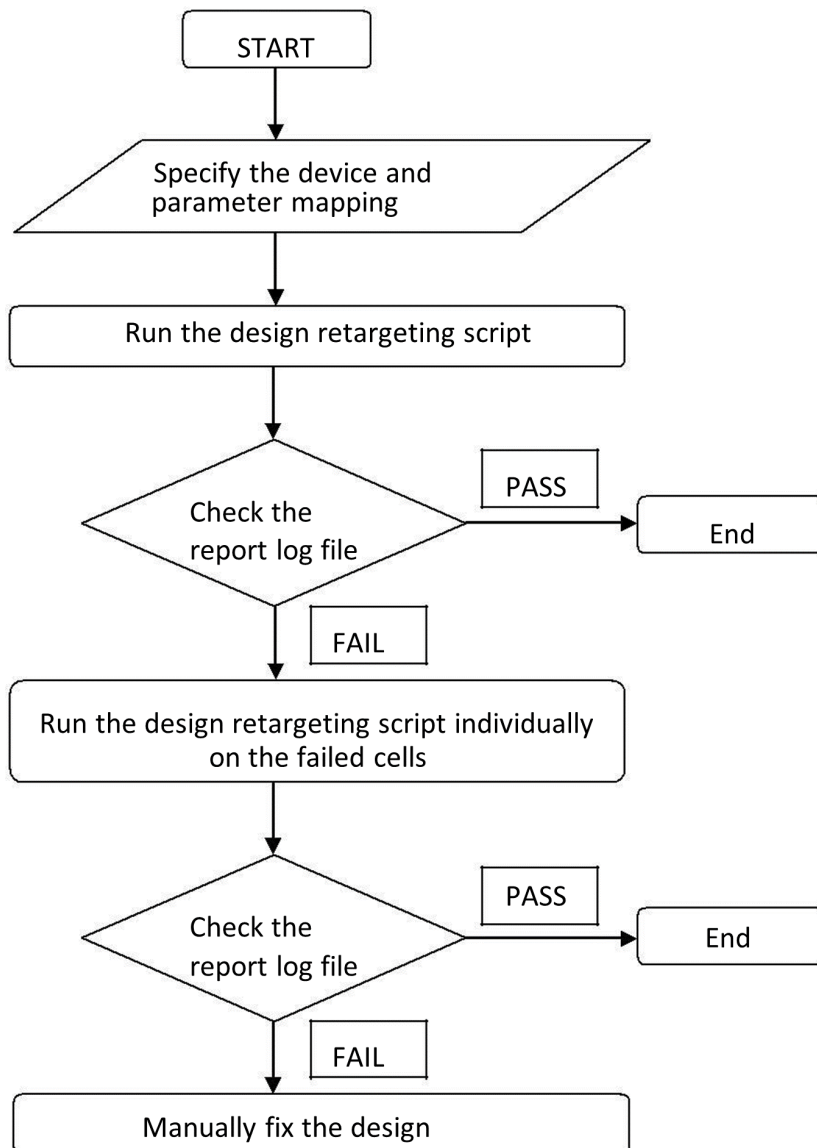
1

Schematic Migration

This topic focuses on converting Custom Compiler schematic designs from one process technology to different technologies.

- [The Schematic Migration Flow](#)
- [Configuring the Setup Tab](#)
- [Specifying Mapping Classes](#)
- [Specifying Information for Cell Mapping](#)
- [Specifying Information for Connecting Terminals](#)
- [Saving the Mapping file and Starting Migration](#)

For schematic designs, the Custom Compiler tool performs the conversion of designs without modifying the symbols while keeping the connectivity intact. The conversion significantly reduces the time with respect to porting the schematic design from one process technology (source technology) to another technology (target technology).

Figure 1 Schematic Migration Flow

The Custom Compiler Schematic Migration flow automates most of the process by

- Customizing cell view and parameter mapping information to map designs from any PDK to any other PDK
- Mapping schematic symbol references to their nearest equivalents
- Mapping instance parameters to their nearest equivalents

- Triggering CDF callbacks to update parameter dependencies
- Removing unused instance properties

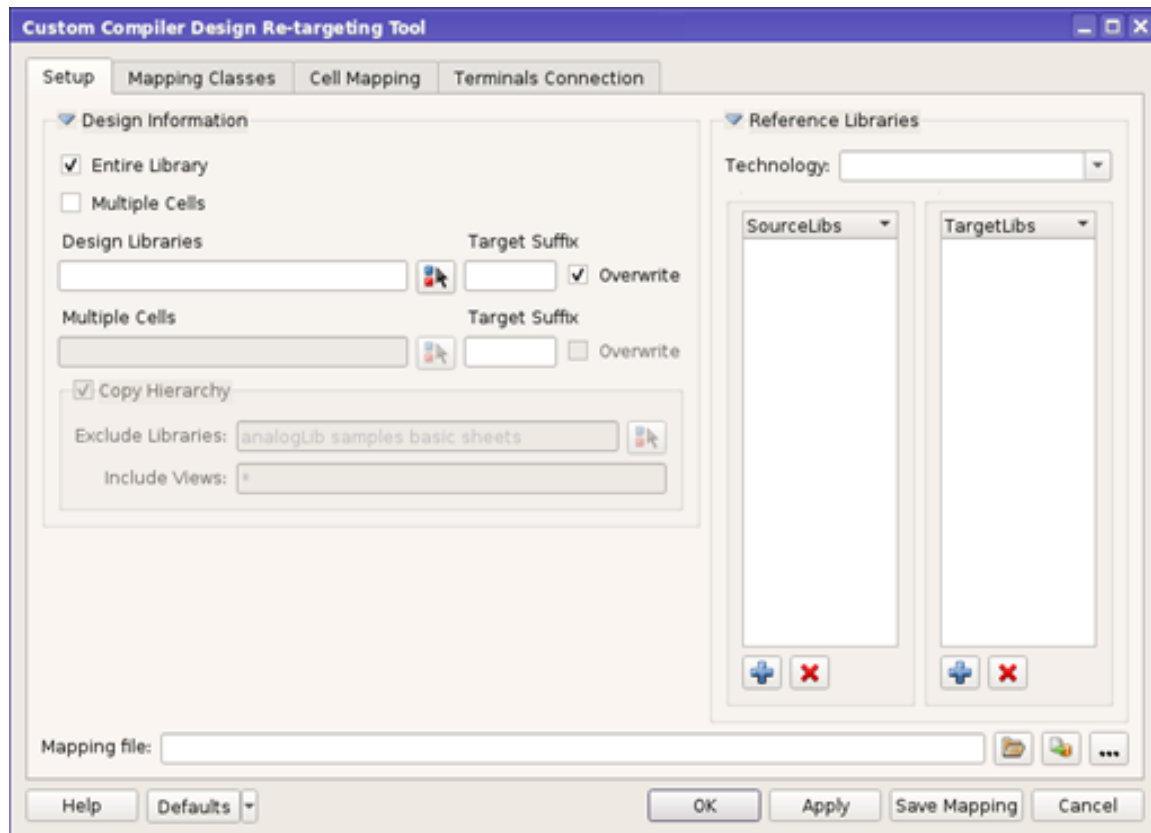
The Schematic Migration Flow

To use the Schematic Migration flow, choose **Tools > Schematic Migration**.

The **Custom Compiler Schematic Migration Tool** dialog box appears. The options in the dialog box provides settings to control the source and target PDK libraries, which design library and schematic cell views to process, and the name of the new, migrated design library to be created.

Configuring the Setup Tab

You need to specify top-level information about the design migration in the **Setup** tab.

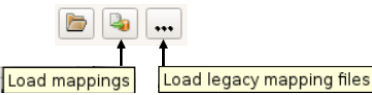


In the **Custom Compiler Schematic Migration Tool**, click on the **Setup** tab and specify the top-level information for design migration as described in [Table 1](#).

Table 1 Sections in the Setup Tab

Section	Item
Design Information	You can run the Schematic Migration flow either by selecting Entire Library or Multiple Cells. Select Entire Library to convert all the cells in the specified library and specify the following information. Design Libraries: Click the Select the Libraries button to open the dialog box and select the design library that you want to convert. Target Suffix: Specify a suffix to add while making the copy of the specified design library. The specified suffix is appended to the design library name. The conversion script runs only on the copied library to ensure that the original data is not lost. For example, if the design library name is <i>test</i> and the target suffix is set to <i>_new</i>, then the copied library name is <i>test_new</i>. (Optional) Overwrite: Select the option only if the copied library already exists in the target library. Select Multiple Cells to run the script on the specified cells only in the given library. First, perform the steps 1 and 3 listed for Entire Library and then specify the information in the following fields: Multiple Cells: Click the Select the Cells button to open the dialog box and select the cells from the specified design library. Target Suffix: Specify a suffix to add while making a copy of the specified cells. (Optional) Overwrite: Select the option only if the copied library cells already exists in the target library. Copy Hierarchy: Select to copy the hierarchy of the migrated cells to the target library. Exclude Libraries: Click the Select the Libraries to Exclude button to open the dialog box and select the libraries that should be excluded while copying. Include Views: Lists the views that should be copied. By default, * indicates that all views are considered.
Reference Libraries	Specify the source and target reference libraries that must be mapped and then specify the following information: Technology: Select the design from the list. SourceLibs: Click the Add button to open the Add Source Libraries dialog box and select the source reference libraries to build the specified design library. Ensure to select only those source reference libraries whose cells must be mapped with the target reference library. TargetLibs: Click the Add button to open the Add Target Libraries dialog box and select the target reference libraries to which the source reference libraries cells must be mapped. (Optional) Use the Delete button to remove the library from the list.

Table 1 Sections in the Setup Tab (Continued)

Section	Item
Mapping File 	Specify and load setup information to map classes and cells in the Mapping Classes and Cell Mapping tabs. Mapping classes refers to parameter mapping of the devices, and cell mapping refers to device and port mapping information Note: The mapping file format is .yaml. Loading the mapping information overrides the existing information in the Mapping Classes and Cell Mapping tabs. To load setup files, • Click on the folder icon and select a .tech_map file that you want to upload, click Select and click Load Mappings to upload the file. Or • Click Load legacy mapping files to open the dialog box and specify and load setup files (.xml) in the Mapping Classes and Cell Mapping fields. This allows you to load information from the old format xml files (mappingClass.xml and cellMapping.xml)

Specifying Mapping Classes

You can create a class for a group of similar type of devices in the **Mapping Classes** tab. Each class contains the parameter mapping information specific to a group of devices. The mapping classes act as information of parameter mapping and are used while defining mapping of cells.

In the **Custom Compiler Schematic Migration Tool**, click on the **Mapping Classes** tab to specify **Classes**, **Parameter Mapping**, **Local Settings for Class**, and **Global Settings** information as explained in [Table 2](#).

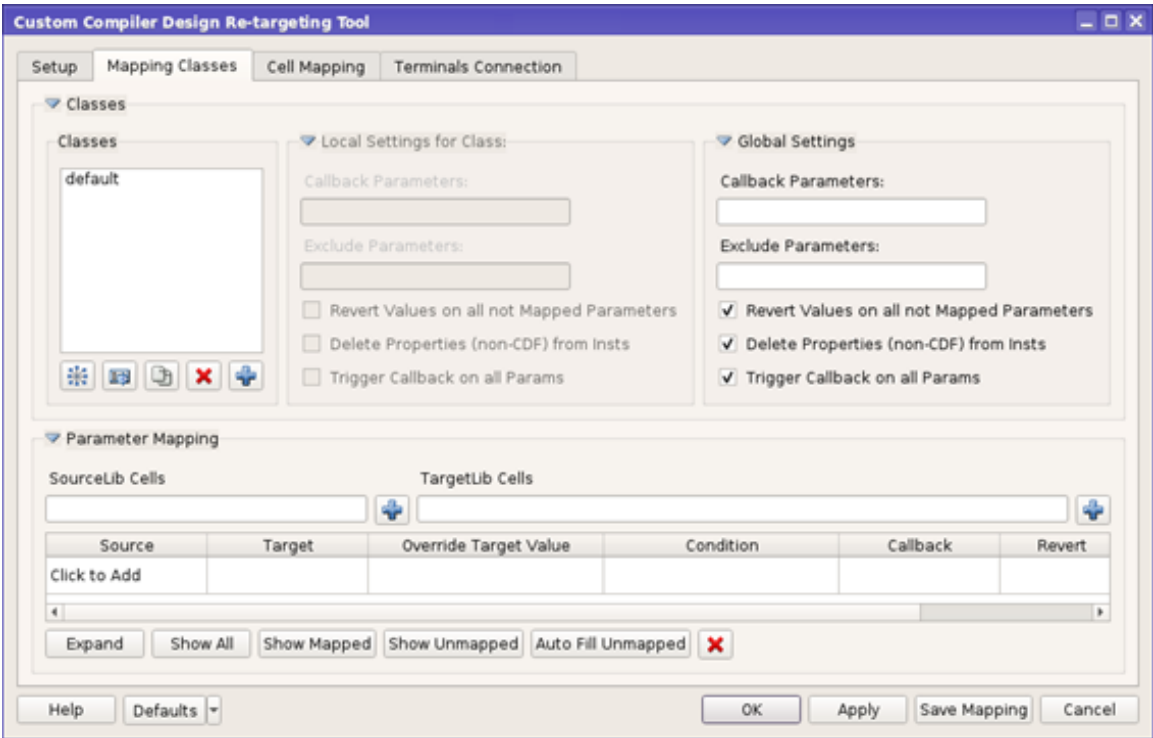
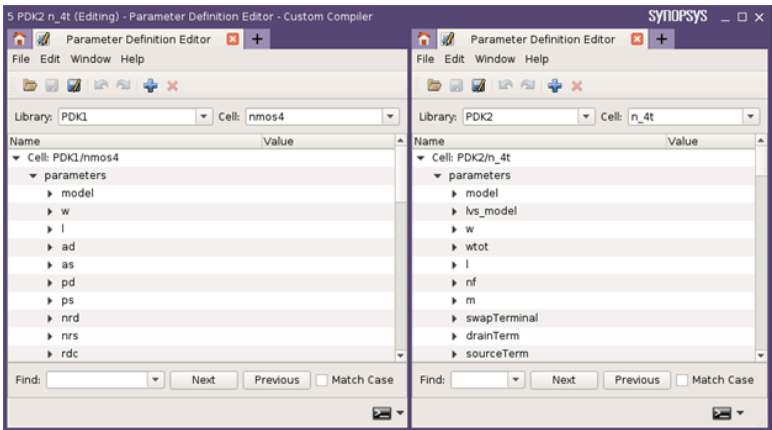


Table 2 Sections in the Mapping Classes Tab

Section	Item
Classes 	Includes the Create, Rename, Copy, Delete, and Add-View Cells button. Click Create to create the class with the word class followed by a number. For example, class1, class2, and so on. The number automatically increments based on the existing class. For example, if class3 already exists, then the class with the name class4 is created. After you create a class, you can add source and target cells to the class using the Add-View Cells button. To add source and target cells to a class, 1. Select a class from the Classes list for which you can add the source and target cells. 2. Click Add-View cells. The Add Cells dialog box opens only if you have configured reference libraries in the Setup tab. 3. Select reference libraries from the Libraries list. Cells corresponding to the selected library appear in the Cells list. 4. Select the cells from the Cells list and click on Add to Source or Add to Target to add the cells that you want from the source and target libraries respectively. 5. Click Ok to add the source and target cells to the selected class. Note: • By default, the default class exists under Classes without parameter mapping information. • When you create a copy of the existing class by clicking Copy, the newly copied class inherits all the settings of the source class.
Local Settings for Class	Provides flexibility to manage parameter mapping for the selected classes only. The field is enabled only when you select a class from the Classes list. Note that the field is disabled if you select multiple classes.. The following fields provide additional flexibility with respect to managing parameter mapping. It applies only to the selected classes. • Callback Parameters: Allows you to specify the list of target cell parameters for which the callback must be triggered. This callback trigger is done after mapping the parameters. This is helpful in the cases when single parameter callback trigger updates all parameters. For example, with Synopsys developed design kits, the callback for all parameters can be triggered with a single parameter called m. • Exclude Parameters: Provides flexibility to exclude target cell parameters, which need not be mapped. For example, model, spice_model, and entryMode. • Revert Values on all not Mapped Parameters: Reverts to the default value of all unmapped parameters. • Delete Properties (non-CDF) from Insts: Allows you to delete parameters from the instance. During parameter mapping, any source parameter (that does not exist in the target parameter or not mapped) is stored as oaProp on the instance. The oaProp is listed under the properties section of Property Editor. If you select Delete Properties (non-CDF) from Insts, these properties are deleted.

Table 2 Sections in the Mapping Classes Tab (Continued)

Section	Item
	• Trigger Callback on all Params: Allows you to explicitly trigger the callback on all the parameters. Using this option might cause increase in the migration time.
Global Settings	Applies to all classes listed in the Classes list. For description of fields, see Local Settings for Class .
Parameter Mapping	Allows to map parameters between source and target parameters. Note: • The mapping information is stored in the class only if you have selected the class from the Classes list. • When the parameter names differ between the source and target cells, you must ensure to map the source and target parameters. Otherwise, the final netlist of the design can be incorrect. Consider the following figure, the target cell uses the <i>wtot</i> parameter for netlisting and the source cell uses the <i>w</i> parameter. For this reason, the parameter with different names should be mapped.



The following information must be specified for mapping parameters. By default, the parameters with the same name from the source and target cells are automatically mapped if they are not specified in the parameter mapping table.

SourceLib Cells and
TargetLib Cells

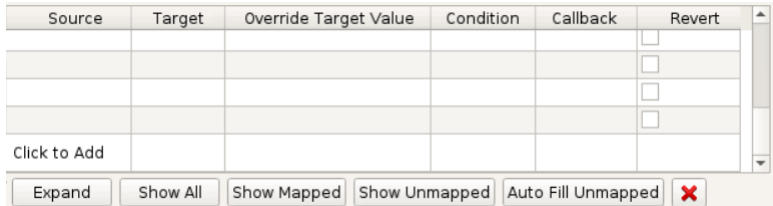
Steps are the same to select and add source and target library cells.

Click **Add** () to open the **Add Source Cell** or **Add Target Cell** dialog box respectively. Select library from the **Library** list and select the cell that you want to map and click **OK**.

The selected cells are added in the `cellName(library Name)` format. Ensure to add cells from both the source and target libraries.

(Optional) Double click in the **Click to Add** (displayed under **Source** in the table) field to manually enter parameter name if you know which parameter must be mapped.

Table 2 Sections in the Mapping Classes Tab (Continued)

Section	Item
Based on the cell selected from the source and target libraries, the Source , Target , and Override Target Value fields are populated in the following table:	
	

Source Lists the name of the source parameter.

Target Lists the name of the target parameter.

Override Target Value Lets you to override the mapped values. The field supports equations based on the source parameter that is evaluated during mapping. Also, supports multipliers like n, u, p, f, m, and so on, providing you with the flexibility to set the parameter of the replaced instances or PCells.

The following syntax lets you to specify equations. Any parameter included in the equation should be within the `$v()` function. For example:

```
3 * 3 * $v(w) / $v(nf)
1u
```

Alternatively, you can use a TCL procedure to define **Override Target Value** with the following format:

```
proc::<procName>
```

You must specify the following arguments with the Tcl procedure:

```
proc procName {inst targetParam srcParamName srcParamValue
oldLCV}
```

Where,

- `inst`: Instance in the target design.
- `targetParam`: Target parameter.
- `srcParamName`: Source parameter name.
- `srcParamValue`: Source parameter value
- `oldLCV`: Library, cell, and view of the source instance.

For example, the `nf` parameter value (see the following table) is set based on the value of the `cellName` parameter value of the target instance. [Example 1](#) shows how to specify this parameter using the Tcl Procedure.

Source	Target	Override Target Value
nf	nf	proc::setParams

Table 2 Sections in the Mapping Classes Tab (Continued)

Section	Item
	Example 1 Using Tcl Procedure <pre> proc setM {inst targetParam srcParamName srcParamValue oldLCV} { set targetParamValue \$srcParamValue set cellName [db::getAttr inst.cellName] switch \$cellName { "nfet" { set targetParamValue [expr \$srcParamValue*4] } "lvnfet" { set targetParamValue [expr \$srcParamValue*8] } } return \$targetParamValue } </pre>
Condition	Lets you to specify conditions based on source parameters, which supports single and multiple conditions. During mapping, also allows you to control mapping of parameters for certain instances based on a few conditions. Note the following points to specify conditions: - Each condition should be enclosed within parentheses ("") separated by a space. - Condition within parentheses should be in the following format: <i>Parameter_Name condition value</i> For example: <pre> \$v(w) <= "1u" (\$v(w) <= "1u") && (\$v(w) >= "2u") </pre> Note: Parameter mapping and override target value hold true only if the specified condition is true. Otherwise, parameter mapping is ignored.
Callback	Defines the order of parameter callbacks to be triggered. The number you specify defines the order of callback triggering. The lower the number, the higher is the priority of that parameter callback. Note: Using this option might increase the migration time.
Revert	Lets you to select only to set the parameter value to its default.

GUI Controls: Buttons to ease mapping of parameters.



Expand	Expands the source parameter combo box list to automatically fill each parameter, listed in the Source field and maps the source parameter with the Target parameter if the parameter names are the same. When you click the Expand button, this button is disabled and the Show Mapped and Show Unmapped buttons are enabled.
Show All	Shows all parameters.
Show Mapped	Shows only the mapped parameters from the expanded list. When you click the Show Mapped button, this button and the Show Unmapped button are disabled and the Expand button is enabled.
Show Unmapped	Shows only the unmapped parameters from the expanded list. When you click the Show Mapped button, this button and the Show Mapped button are disabled and the Expand button is enabled.
Auto fill Unmapped	Tries to map unmapped parameters based on the parameter names.
Delete	Lets you to select and delete any row from the parameter mapping table

Specifying Information for Cell Mapping

You must specify cell mapping after creating the classes with parameter mapping. For cell mapping, you must provide the information about the source and target cells that must be mapped. During the Schematic Migration flow,

- For schematic designs, swapping of ports is not allowed. For example, when swapping the connections between gate and drain of the MOSFETs.
- Target symbol size should not be larger than the source symbol size; otherwise, it might cause wiring issues.

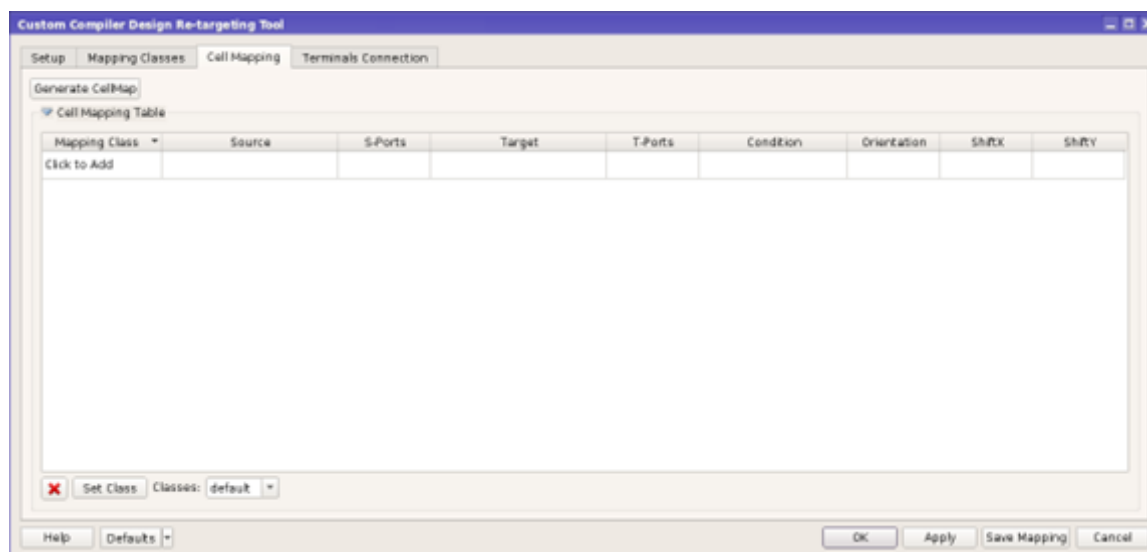


Table 3 lists the fields for the cell mapping table and see Figure 2.

Table 3 Fields in the Cell Mapping Tab

Fields	Description
Mapping Class	Click to view the list of available classes that were created in the Mapping Classes tab. Choose the appropriate class.

Table 3 Fields in the Cell Mapping Tab (Continued)

Fields	Description
Source	Click to view the list of source libraries, cells, and views based on the following settings and choose the appropriate source libraries, cells, and views that should be mapped: • Class from Mapping Classes If you have added the source cells into the selected class using Add-View cells in the Mapping Classes tab, then the libraries, cells, and views list is driven by this setting. • Reference libraries from Setup If the selected class does not contain any source cells, then the libraries, cells, and views list is driven by the source reference libraries that are set in the Setup tab. This means all the cells from the source reference libraries are listed.
S-Ports	Automatically updated with the selected source cell ports. You must ensure that the order of the ports match with the target ports, T-Ports. If there is a mismatch in the port order, then you can manually correct either S-Ports or T-Ports to ensure correct matching.
Target	Similar to the Source field. Click to view the list of target libraries, cells, and views based on the following settings and choose the appropriate target libraries, cells, and views that should be mapped: • Class from Mapping Classes If you have added the target cells into the selected class using Add-View cells in the Mapping Classes tab, then the libraries, cells, and views list is driven by this setting. • Reference libraries from Setup If the selected class does not contain any source cells, then the libraries, cells, and views list is driven by the target reference libraries that are set in the Setup tab. This means all the cells from the target reference libraries are listed.
T-Ports	Automatically updated with the selected target cell ports. You must ensure that the order of the ports match with the source ports, S-Ports. If there is a mismatch in the port order, then you can manually correct either S-Ports or T-Ports to ensure correct matching.

Table 3 Fields in the Cell Mapping Tab (Continued)

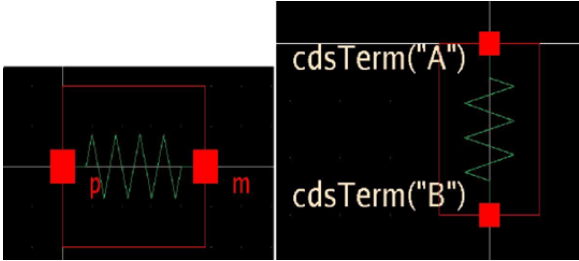
Fields	Description
Condition	Map the cells or device-based on the parametric conditions. For example, If the width of MOS is between 1 u to 2 u, then map the cell to normal MOS. If the width is greater than 2 u, then map the cell to high voltage MOS. You can specify conditions based on source parameters, which supports single and multiple conditions. During mapping, provides you the flexibility to control mapping of parameters for certain instances based on a few conditions. Note the following points to specify conditions: • Each condition should be enclosed within parentheses (") separated by a space. • Condition within parentheses should be in the following format: <i>Parameter_Name condition value</i> For example: <pre>\$v(w) <= "1u" (\$v(w) <= "1u") && (\$v(w) >= "2u")</pre> Note: Parameter mapping and override target values hold true only if the specified condition is true. Otherwise, parameter mapping is ignored.
Orientation	Require when the original symbol orientations and target symbol orientations are different. The default is R0. In the following figure, the source and target cells are different in their orientation by 270-degrees. This field allows you to have their schematic intact without modifying the target symbols. 

Table 3 Fields in the Cell Mapping Tab (Continued)

Fields	Description
ShiftX and ShiftY	Require when the origins of the original and target symbols are different. The default is 0. In the following figure, the source symbol origin is on minus terminal and the target symbol origin is on plus terminal. To match the origin, the ShiftX and ShiftY options must be specified. As the distance between the plus and minus terminals of source cell is 1 unit in y-direction, ShiftY = 1. Since, the x-coordinates are the same, ShiftX = 0.

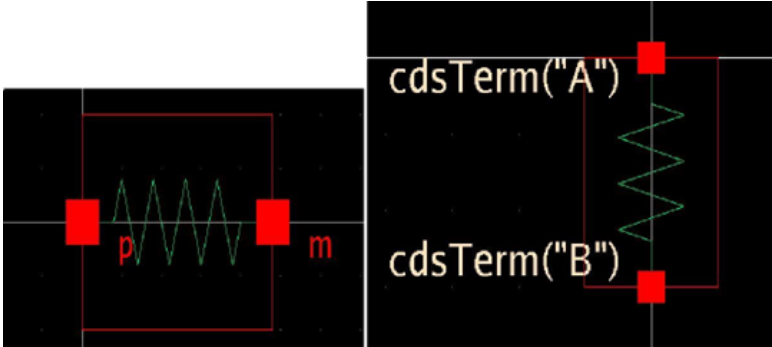
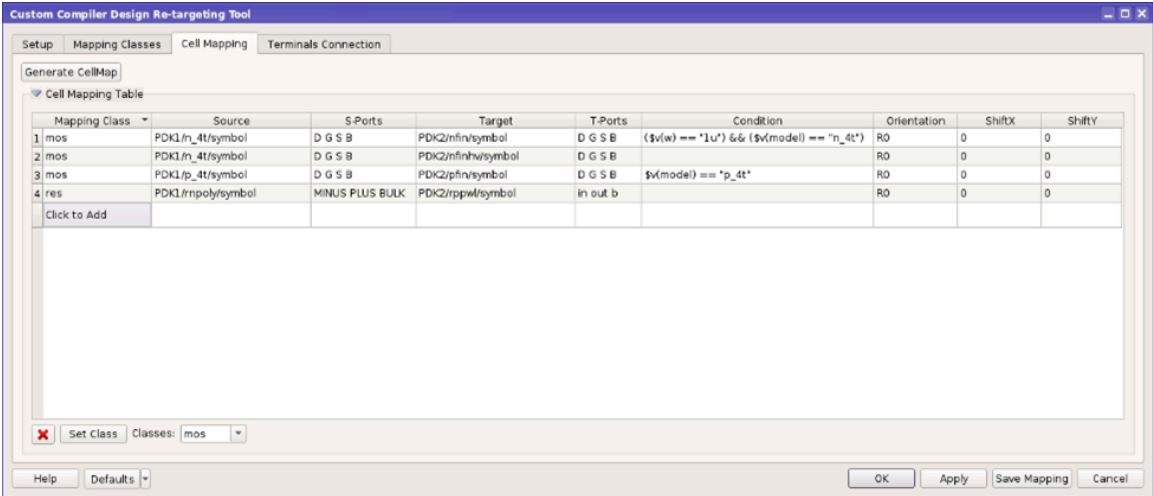
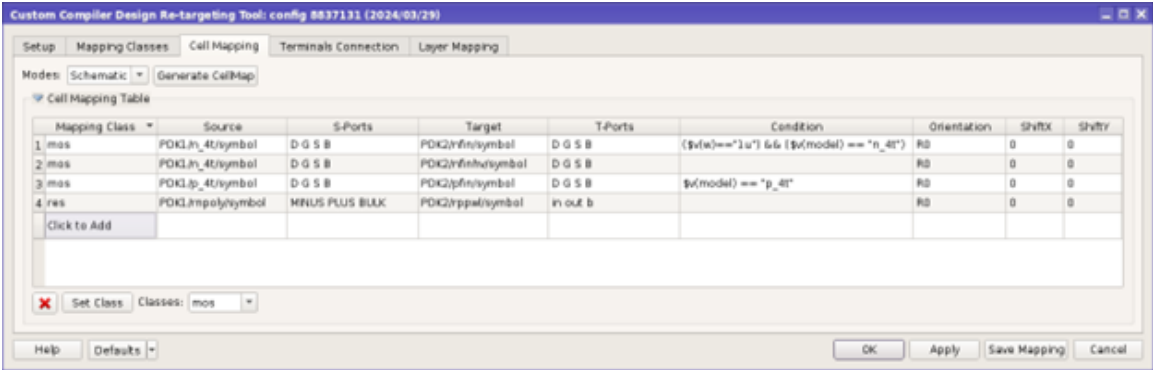


Figure 2 Cell Mapping Updated With Information



Chapter 1: Schematic Migration

Specifying Information for Cell Mapping



Specifying Information for Connecting Terminals

You must specify information about connections to unrouted instance terminals.

Figure 3 *Terminal Connection Updated With Information*

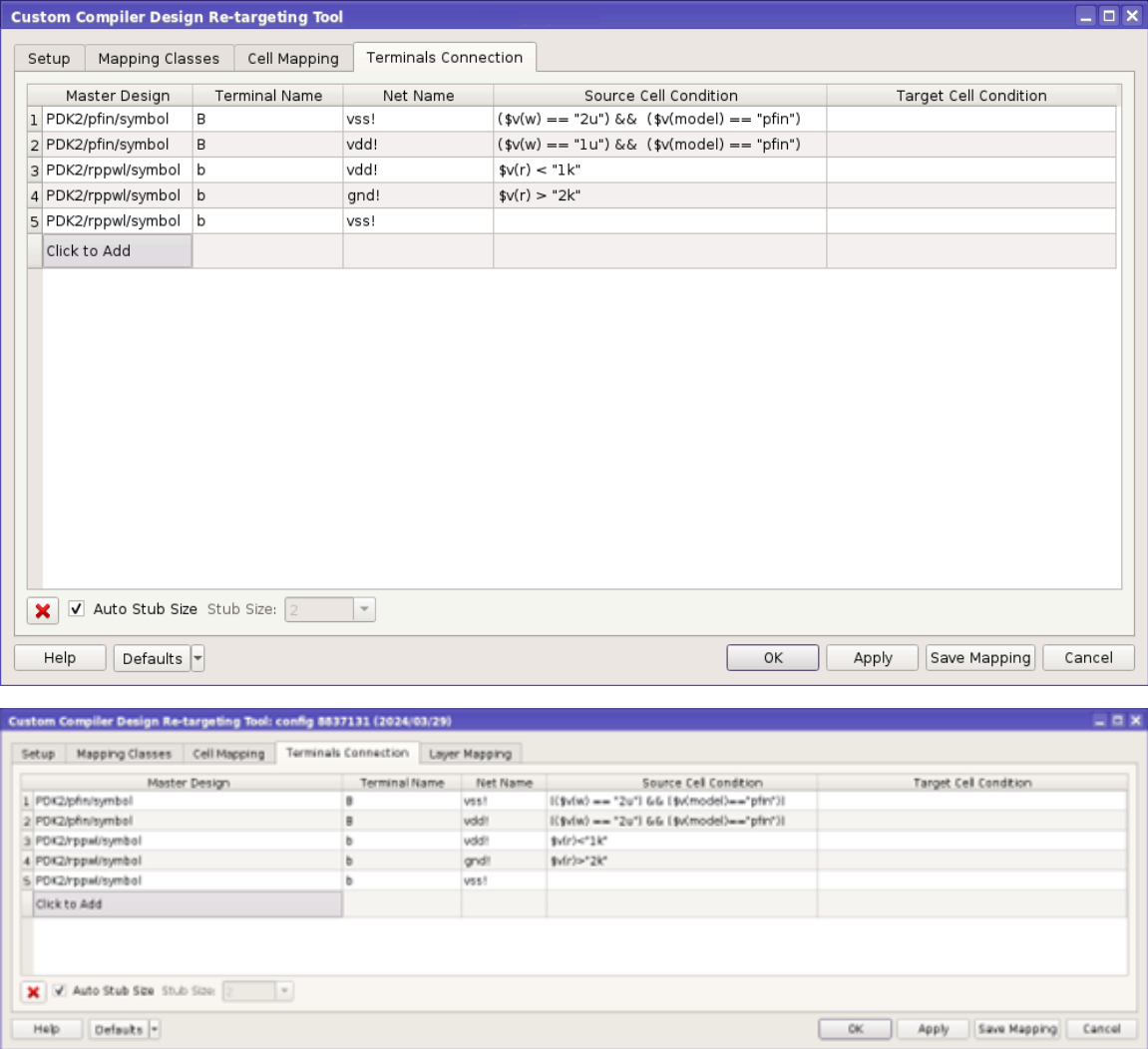


Table 4 lists the fields for the terminals connection table and see Figure 3.

Table 4 *Fields in the Terminals Connection Tab*

Fields	Description
Master Design	Click to view the libraries, cells, and views list and choose the appropriate source libraries, cells, and views.

Table 4 Fields in the Terminals Connection Tab (Continued)

Fields	Description
Terminal Name	Choose the terminal name from the list.
Net Name	Specify a net name.
Source Cell Condition and Target Cell Condition	Connects terminal to a specific net-based on parametric conditions. For example, If the width of MOS is 1 u and mode name is <i>pmos4</i>, the connects the terminal B to the <i>vdd!</i> global net. You can specify conditions based on source and target parameters, which supports single and multiple conditions. During mapping, provides you with the flexibility to control mapping of parameters for certain instances based on a few conditions. Note the following points to specify conditions: - Each condition should be enclosed within parentheses ("") separated by a space. - Condition within parentheses should be in the following format: <code>Parameter_Name condition value</code> For example: <code>\$v(w) <= "1u"</code> <code>(\$v(w) <= "1u") && (\$v(w) >= "2u")</code> Note: Parameter mapping and override target value hold true only if the specified condition is true. Otherwise, parameter mapping is ignored.
Auto Stub Size	Select to set the stub size automatically.
Stub Size	Deselect to manually specify the stub size. Add a stub from 3-ports to 4-ports device to the unconnected port when migrating, allowing only to modify the stub size.

Saving the Mapping file and Starting Migration

After you complete defining mapping for schematic designs, you can save the mapping at any point of time by clicking on **Save Mapping** to create the *migration_mappings.tech_map* file. The technology mapping file contains all the information of the created classes in the **Mapping Classes** and **Cell Mapping** tabs.

Starting Migration

After creating the classes and defining the cell mapping, you can start the migration by clicking on **OK**. Migration involves the following processes:

- Saving the mapping information in the technology mapping file, *migration_mappings.tech_map* file.
- Starting the migration and creating the *report<mode>_<date>_<time>.log* file to monitor the progress of migration and to view the information on the migrated cells.

Preserving Information After Migration

After migration, the following changes are made in the schematic designs:

- All wires with wire names are preserved.
- Connectivity check is run on the cell before and after the conversion. Mismatches are reported in *reportSchematic_<date>_<time>.log* file and the cell conversion process is treated as failed.

2

Layout Migration

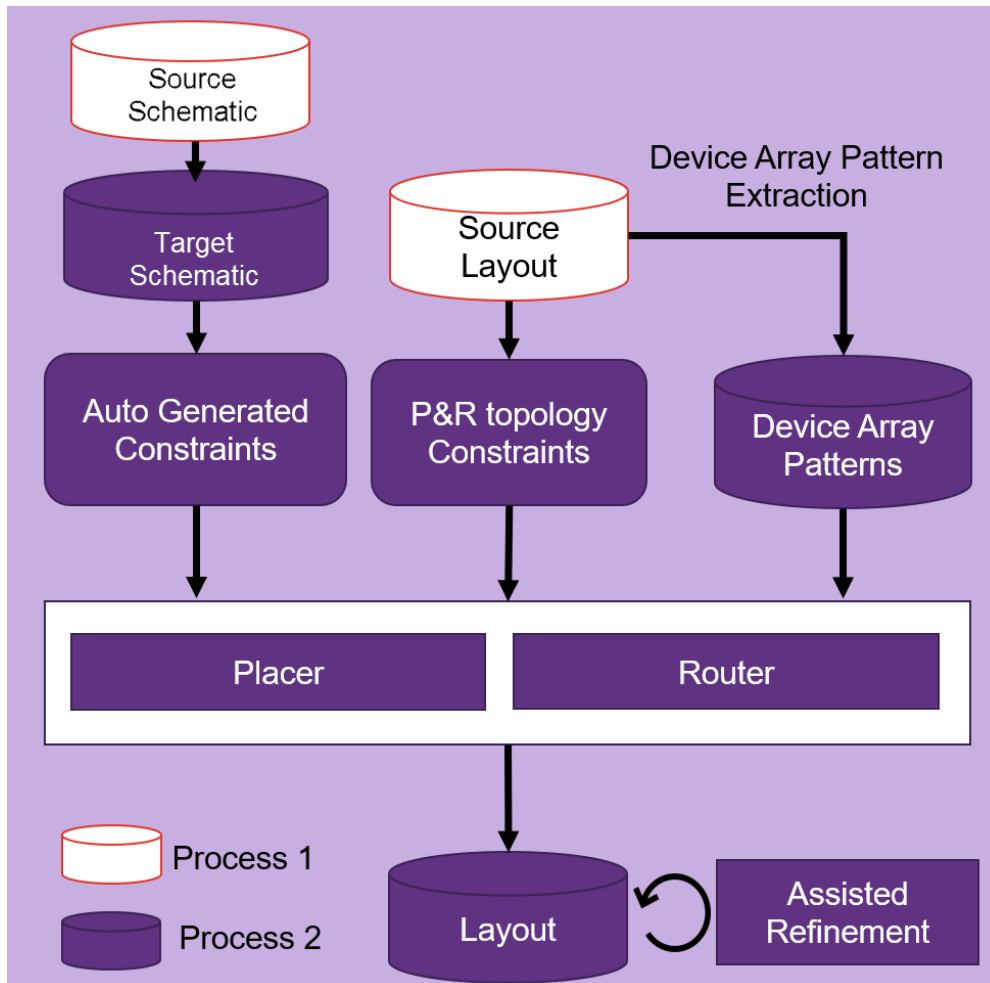
Intellectual Property and Design reuse is an integral part of modern circuit design and implementation methodologies. The intent is to reduce design time, reduce iterations between initial design concept to have the final design with complete layout ready for tapeout. In addition, at advanced nodes, the impact of parasitics increases making it a requirement to incorporate layout effects as quickly as possible to consider them when finalizing the design schematics.

Approaches used by design teams for custom designs have been varied and built organically for their own design styles. The Synopsys solution to layout migration is design and technology node independent, relying on several steps that are generalized as such:

- Capture the source design layout intents
- Create the layout view through SDL from the optimized schematic view
- Load the source design intents
- Apply placement and routing layout regeneration

The following figure shows the design migration workflow.

Figure 4 Design migration workflow



The following sections describe each of the steps in the design migration process.

- [Reviewing the Source Design](#)
- [Preparing the Layout Parameters Configuration file](#)
- [Extracting the Source Design Using Design Migration](#)
- [Technology Map File Review and Import](#)
- [Generating Target Design - Setup](#)
- [Generating Target Design- Pin Placement](#)
- [Generating Target Design- Routing](#)

- [Generating Target Design– Pin Generation](#)
- [Comparing the Migrated Design with Source Design](#)

Note:

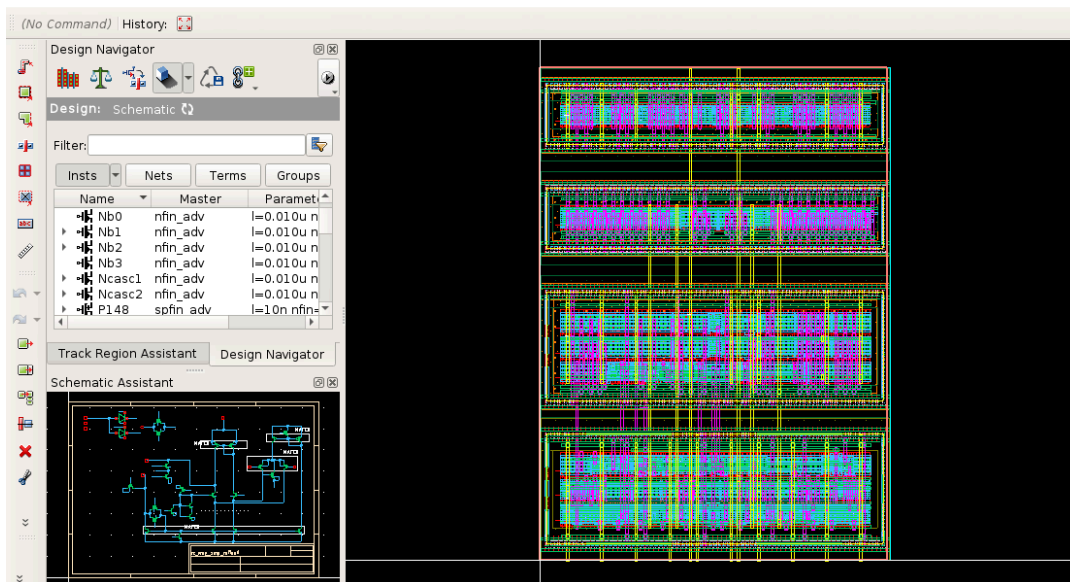
The figures are not representing any specific node PDK, so names may be different in your environment when using a foundry PDK and their own design.

Reviewing the Source Design

Perform the following steps to review the source design.

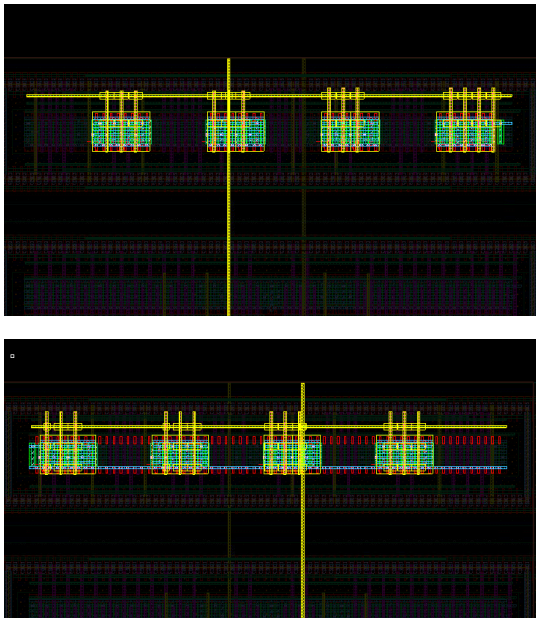
1. Open the source layout design with SDL. To open the design with SDL, right-click on the **layout** view and select **Open with SDL**.
2. Review **Design Navigator** to see if all instances from schematic are realized and all nets are routed without opens or shorts.

Figure 5 Source design in SDL layout 1



3. Cross highlight layout with **Schematic Assistant** instances and mark notes **MATCH**.
Observe the matching transistor arrays in the layout design. Notice the way they are created is in symmetry.

Figure 6 Symmetrical instance and their drain routing



4. Query an instance in the **Property Editor** form, and enable **** Show Term & Via Options ****.

Observe the **Left terminal** and **Right terminal** parameters to control pin layers of the transistor's left side and right side respectively. These two parameters' values need to be swapped when such an instance is placed in *MY* orientation after migration.

Figure 7 Review instance parameter with Property Editor

** Show Term & Via opti...	☒	
S/D metal orientation	Horizontal	
Source term color		
Drain term color		
Gate top term color	None	
Gate bottom term color	None	
Terminal width	22n m	
Left terminal	DIFFCONT2	
Inner terminals	DIFFCONT2	
Right terminal	DIFFCONT2	
Swap Terminals	☐	
Swap Hor. Terminals	☐	
VIA0 type	Standard	

5. In **Property Editor**, look for the **gate_via_locs** parameter, which is used to store gate via size and position. The value of this parameter has to be changed during migration to reflect the corresponding via dimension and locations in the new process node.

Figure 8 Review instance parameter with Property Editor

Metal Tracks	(0.055 red 0.022) (0.105...
source_via_locs	(0.155 red 0.022)
gate_via_locs	(0.055 red 0.022)
drain_via_locs	(0.255 red 0.022)
fixedCellHeight	360n
gateCutWidth	24n
leLDEHalo	5

6. Close **Property Editor** without applying changes and **Close** the design.

Preparing the Layout Parameters Configuration file

The Layout Parameters Configuration file is an ASCII file that defines the cells and parameters to be extracted and reused in YAML format. The layout parameters configuration file is an input for source design extraction in the Design Migration flow.

Perform the following steps to create and review the layout parameters configuration file.

1. Open the `configParamExtraction.yaml` file in a text editor.
2. For **lib**, enter the PDK library name- the source design.
3. For **cell**, enter the cells to be extracted.
4. For **parameter**, enter the parameters to be extracted and reused.
5. For **leftRightParamPair**, specify the left-right parameter pairs that would have to swap when transistor is placed mirrored.
6. For **syncDummy**, specify **true** so that the dummy device parameters are refers to the adjacent active devices.
7. Save and close layout parameters configuration file.
8. (Optional) Open the reference file provided in the YAML format and validate the content.

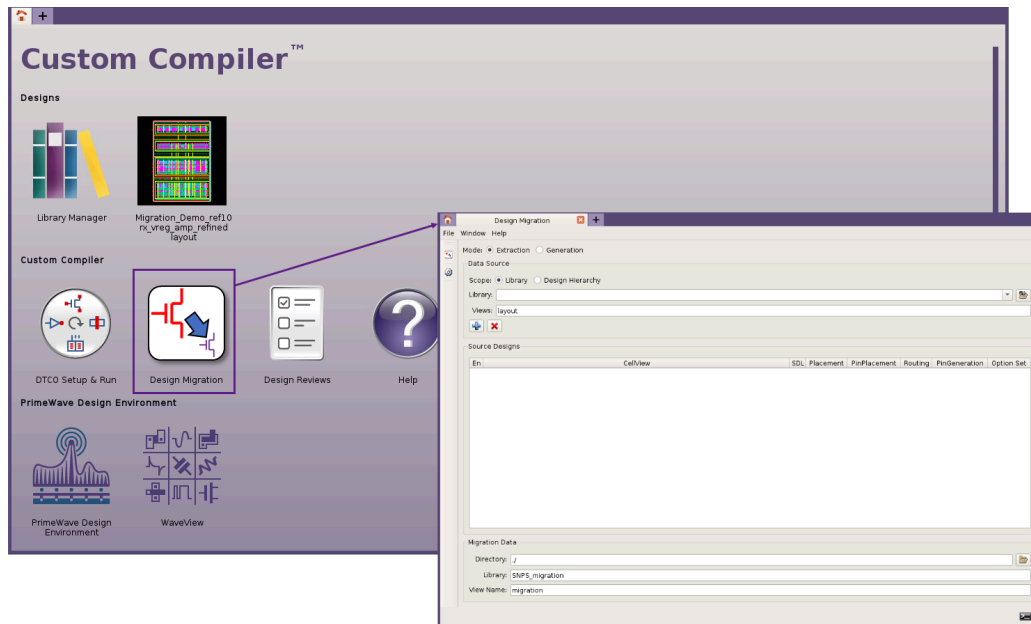
The following is an example of the content in a Layout Parameters Configuration file:

```
configParameterExtraction:
  - lib: reference10
    cell:
      - nfin
      - pfin
      - snfin
      - spfin
      - nfin_adv
      - pfin_adv
      - snfin_adv
      - spfin_adv
    parameter:
      - leftTerm
      - rightTerm
      - gate_via_locs
    leftRightParamPair:
      leftTerm: rightTerm
    syncDummy: true
```

Extracting the Source Design Using Design Migration

Perform the following steps to extract the source design using the Design Migration GUI tool and review the result.

1. In the Custom Compiler Home page, click the **Design Migration** icon to open the Design Migration window.

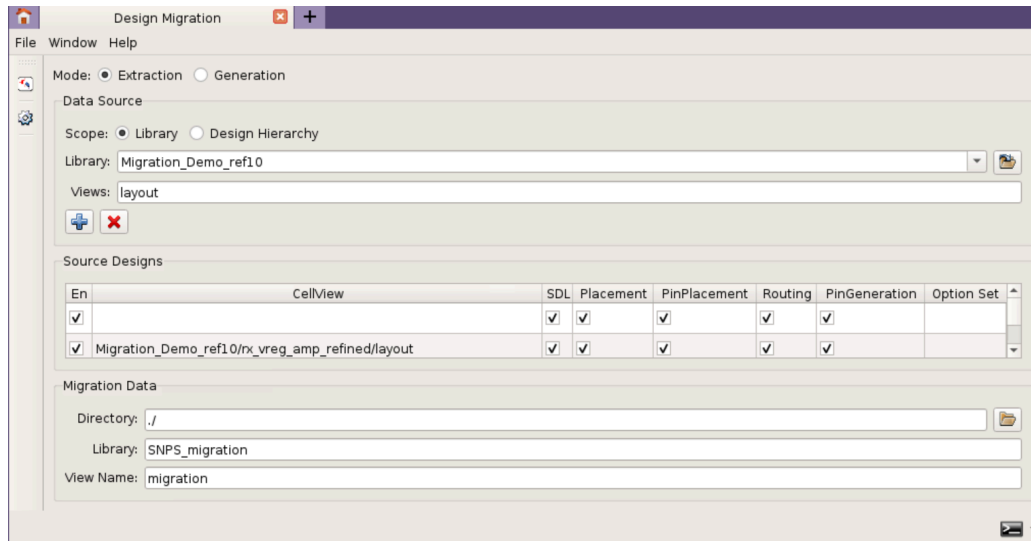


2. **Select Mode: Extraction.**

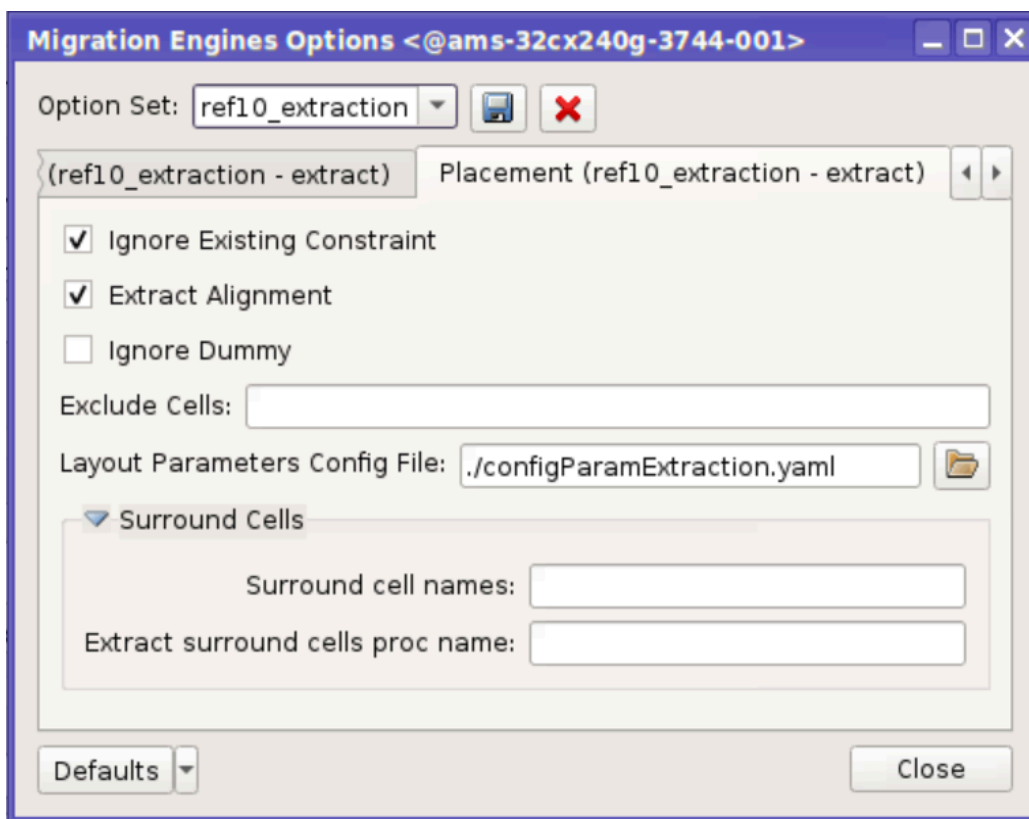
Note:

If the design has not been extracted before, the mode option, **Generation** is not visible.

3. In the **Data Source** section, set the following:
 - **Scope:** select between **Library** or **Design Hierarchy**
 - **Library/Cell/View:** Select the intended library, cell, and layout view.
 - **Exclude:** Specify the exclude list. This option is available only for **Scope: Design Hierarchy**.
4. Click the  button to add view. In the **Source Design** section, the cellview appears.
5. In the **Migration Data** section, specify the storage library for migration data.
 - **Directory:** ./
 - **Library:** <migration library name>
 - **View Name:** migration

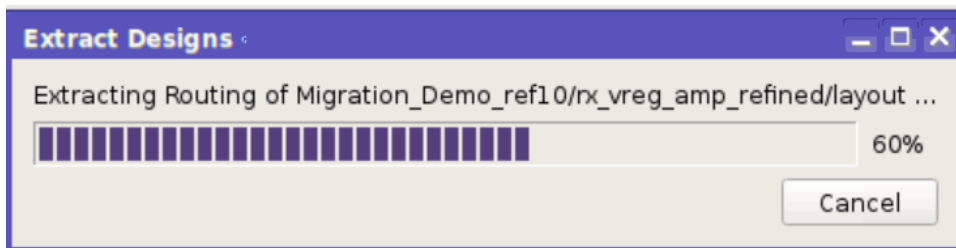


6. Click the  icon to open the **Migration Engines Options** dialog box.
7. For **Option Set**, select an available option set, click the save icon.
8. Click the **Placement** tab and ensure that the **Layout Parameters Config File** is pointing to the file from [Preparing the Layout Parameters Configuration file](#).

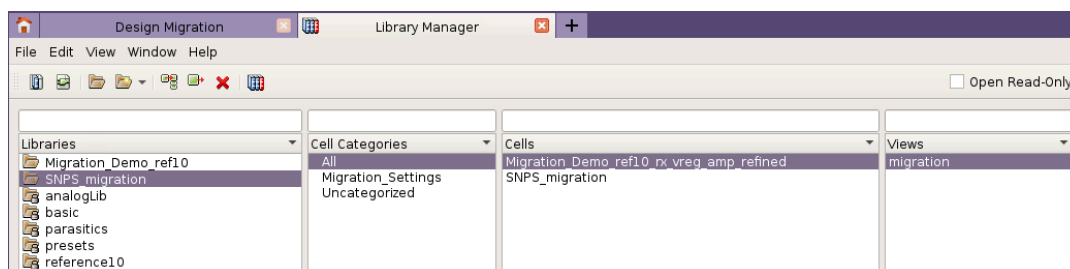


9. Close the **Migration Engines Options** dialog box to go back to the **Design Migration** window.

10. Click the  button to extract designs from the selected library. The **Extract Designs** dialog box appears with the progress bar and percentage.



11. After the extraction is complete, in the **Library Manager** window, notice the migration library listed under **Libraries** as the placeholder of extracted information from design.



12. (optional) Right-click on **migration** under **Views** and select **Open Terminal Here** to open a new xterminal.

Check the file list in the directory and review the YAML file extracted from source design is as follows.

```
unix% ls *.yaml
fig_group_config.yaml migration.yaml pr_config.yaml
layout_context.yaml origin.yaml routing.yaml
layout_param.yaml pin_relative_data.yaml
```

Exit the xterminal after reviewing.

13. Close Custom Compiler.
14. Change directory to the target design working directory and link the migration library in the working directory. Add the library definition entry for the migration library as *lib.defs*, as shown in the following.

```
unix% cd ../<target_design_dir>
unix% ln -s ../source_ref10/<migration_library_name>
unix% echo "DEFINE SNPS_migration ../SNPS_migration" >> lib.defs
```

Technology Map File Review and Import

Perform the following steps to review the content of the Technology Map File for Design Migration and also import it.

1. Open the YAML format Technology Map File with a Text Editor to review the contents.
 - a. Specify the header of the mapping file, **format**.
 - b. Specify Inst matching constraint attributes to control transistor array style. Set instance boundary LPP and abut different rows using this LPP with 0um spacing to have a side by side placement.
 - c. Specify the guard ring constraint to control the PCell master and parameter values.

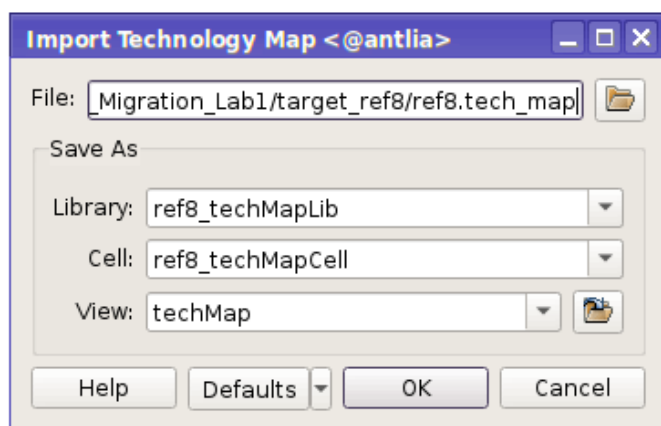
- d. Layout parameter mappings set proper instance values to accommodate target design attached to the new iPDK. The gate contact size and position are controlled using *gate_via_loc* parameter.

The following is an example of the Technology Map file:

```
---
format: CC_Design_Migration
version: 1
minorVersion: 0
ConstraintUpdateInfo:
  cmInstMatching:
    - componentType: mos
      attribute:
        - sizePolicy.policy: none
        - style: textPattern
        - spacingType: micron
        - abutmentStyle: abut
        - boundaryStyle: specify
        - sameRowLPP: "instance drawing"
        - diffRowLPP: "instance drawing"
        - differentRowSpacings: 0
        - odSharing: 1
  cmGuardRing:
    - masterCell: [pfin_adv]
      attribute:
        - master: reference8/MPP_GR_NWell_CPO/layout
        - parameters: "{{polyWidth 0.0008u}}"
    - masterCell: [spfin_adv]
      attribute:
        - master: reference8/MPP_GR_NWell_CPO/layout
        - parameters: "{{polyWidth 0.0008u}}"
    - masterCell: [nfin_adv]
      attribute:
        - master: reference8/MPP_GR_PSub_CPO/layout
        - parameters: "{{polyWidth 0.0008u}}"
    - masterCell: [snfin_adv]
      attribute:
        - master: reference8/MPP_GR_PSub_CPO/layout
        - parameters: "{{polyWidth 0.0008u}}"
layoutParamMapping:
  - paramMap:
      leftTerm: leftTerm
      rightTerm: rightTerm
  - paramMap:
      gate_via_locs: gate_via_locs
      valueMap:
        "(0.305 red 0.022)": "(0.273 red 0.0176)"
        "(0.055 red 0.022)": "(0.047 red 0.0176)"
```

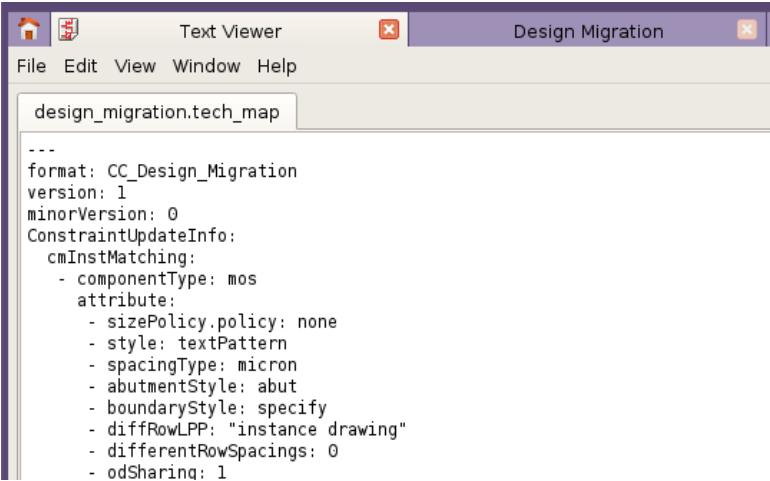
2. Close the technology map file.

3. Make sure your working directory is the target directory and open Custom Compiler.
4. In the Custom Compiler Home page, click the **Design Migration** icon to open the Design Migration window.
5. (optional) Choose **File > Import Tech Map** to open the **Import Technology Map** dialog box. Use the file selector to set the following:
 - **File:** Location of technology map.
 - **Set As:** Select the techMap cellview.
6. Click **OK** to import.



Since, the techMap cellview already exists, the tool prompts to overwrite. Click **OK** to proceed with overwrite and complete the import process.

7. Choose **Library Manager > File > Open** and select the techMap cellview. Right-click on **techmap** and select **Open with Text Viewer** to open the imported technology map file. Review the content is correctly imported as text view, as shown in the following figure.

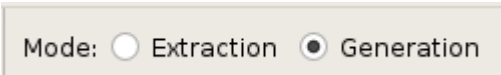


8. Close **Text Viewer** window and go back to the **Design Migration** window.

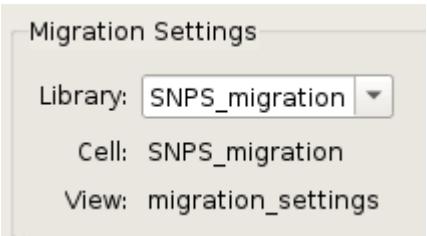
Generating Target Design - Setup

Perform the following steps to setup Generate Target Design using the extracted c and mapping cellview prepared in [Technology Map File Review and Import](#).

1. In the Design Migration window, select **Generation** as the **Mode**.



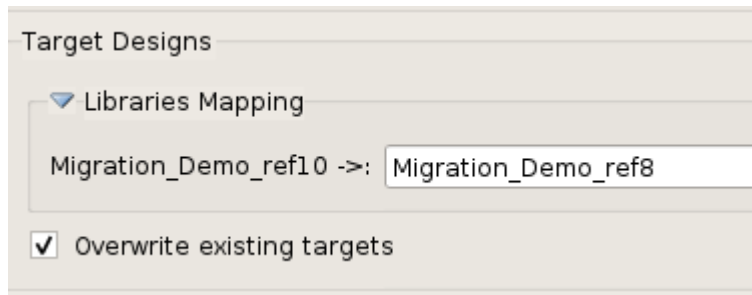
2. In the **Migration Settings** section, select the extracted migration library as the **Library**.



3. The **Source Designs** section displays the extracted designs and timestamp.

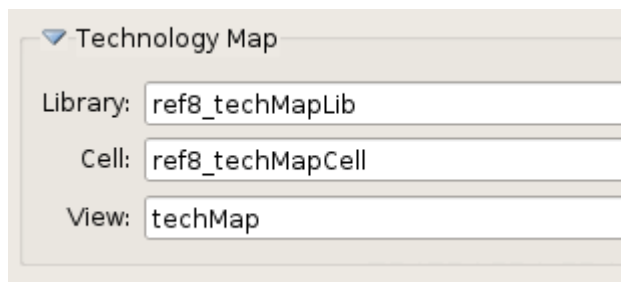
Depth	En	Source	SDL	Placement	PinPlacement	Routing	PinGeneration	Option Set	Overwrite	Timestamp
1	✓	Migration_Demo_ref10/rx_vreg_amp_refined	✓	✓	✓	✓	✓			06/11/2024 03:22:15

4. In the **Target Designs** section, select the **Libraries Mapping**.



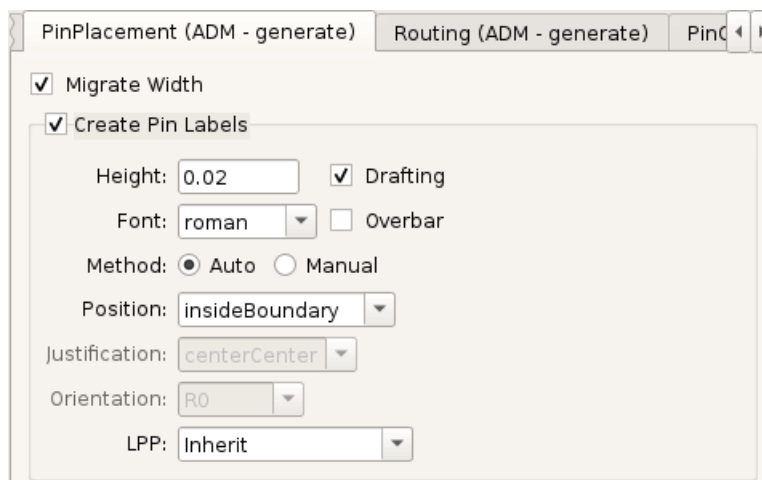
The 'Target Designs' dialog box is shown. It has a section titled 'Libraries Mapping' with a mapping from 'Migration_Demo_ref10' to 'Migration_Demo_ref8'. Below this, the 'Overwrite existing targets' checkbox is checked.

5. Ensure the **Source Directory** option is set.
6. In the **Technology Map** section, set the technology map view imported from previous task.



The 'Technology Map' dialog box is shown. It contains three input fields: 'Library' set to 'ref8_techMapLib', 'Cell' set to 'ref8_techMapCell', and 'View' set to 'techMap'.

7. Click the  button to open the **Migration Engines Options** dialog box. For **Option Set**, select **ADM**.
8. Click the **Pin Placement** tab and ensure the **Create Pin Labels** check box is selected to generate pin labels in migration.



The 'Migration Engines Options' dialog box is shown with the 'PinPlacement (ADM - generate)' tab selected. The 'Migrate Width' and 'Create Pin Labels' checkboxes are checked. Under 'Create Pin Labels', the following settings are visible: Height: 0.02, Drafting checkbox checked, Font: roman, Overbar checkbox unchecked, Method: Auto (selected), Manual (unselected), Position: insideBoundary, Justification: centerCenter, Orientation: R0, and LPP: Inherit.

- Click the **Routing** tab and ensure the **Rebuild Topology** and **Optimize Topology** check boxes are selected to rebuild and optimize migrated routing topologies.

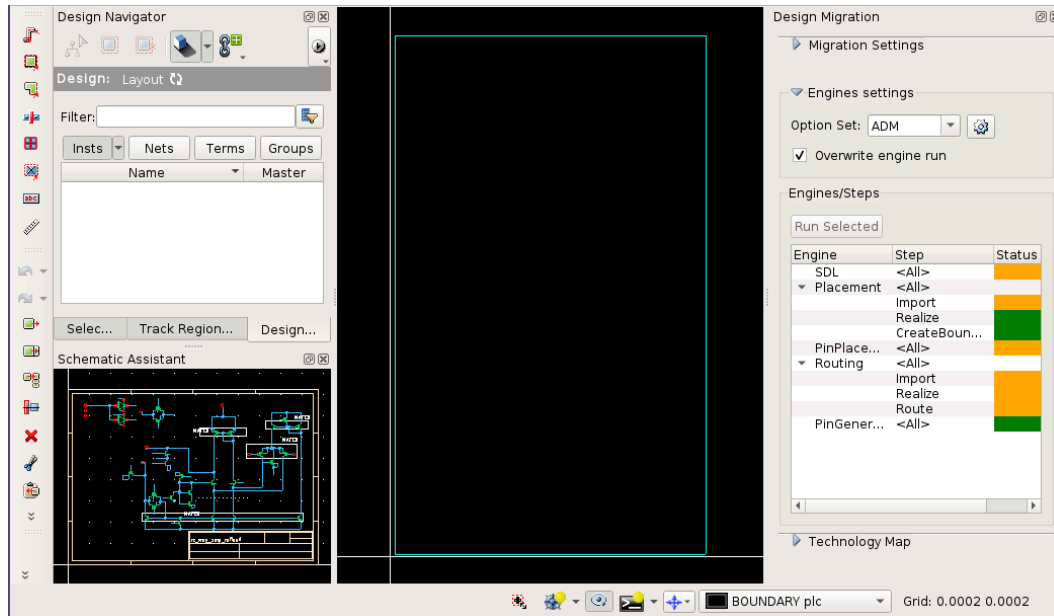
The screenshot shows the 'Routing (ADM - generate)' tab of the 'Migration Engines Options' dialog. The 'Steps' section has 'Import', 'Realize', and 'Route' checked. The 'Import Option' section has 'Rebuild Topology', 'Optimize Topology', 'Migrate Trunk Width', 'Migrate Width as Constraint', and 'Migrate Layer Usage' all checked. The 'Realize Option' section has 'Only Basic Spacing' and 'Realize Short Length' checked, while 'Realize Wrong-Way' is unchecked. The 'Route Option' section has 'Enable Connectivity Engine' checked, while 'Only Basic Spacing' is unchecked.

- Click **Close** to close the **Migration Engines Options** dialog box.
- In the Design Migration window, in the **Source Designs** section, and right-click on the design to select **Generate (Interactive)**. This method of migration is used to interactively generate design and it allows interventions between steps.

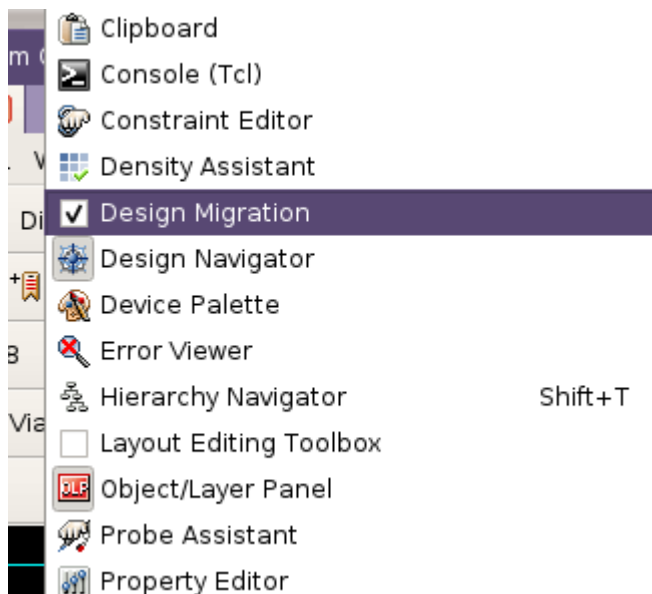
The screenshot shows the 'Source Designs' section of the Design Migration window. It includes a 'Show run statuses' checkbox and a 'Show logs' button. Below is a table with columns: Depth, En, Source, SDL, Placement, PinPlacement, Routing, and PinGeneration. The table has three rows, with the third row highlighted in purple. A right-click context menu is open over the third row, showing 'Generate (Interactive)' and 'Generate Hierarchy' options.

Depth	En	Source	SDL	Placement	PinPlacement	Routing	PinGeneration
1	✓		✓	✓	✓	✓	✓
1	✓		✓	✓	✓	✓	✓
	✓	Migration Demo ref10/rx vreg amp refined	✓	✓	✓	✓	✓

12. Custom Compiler brings the target design with the **Design Migration** assistant docked on the right side of canvas.

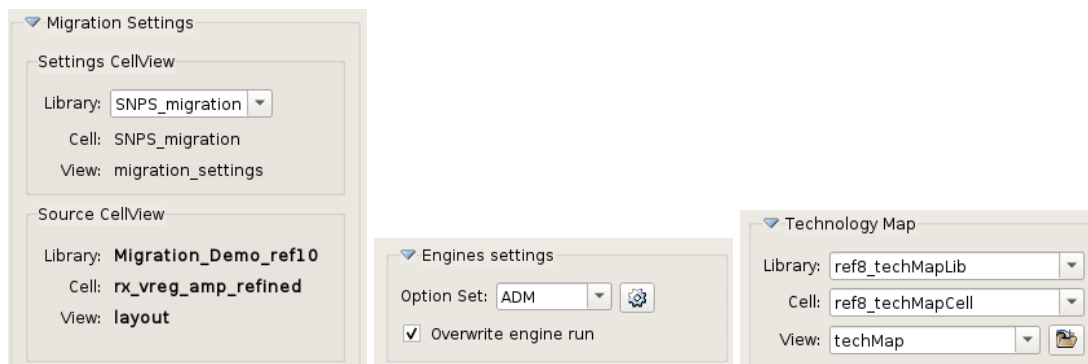


If the **Design Migration** assistant does not show up or is closed by you, right-click on an empty area of the toolbar and select **Design Migration** to bring it back.



Generating Target Design- Pin Placement

1. Open the layout cellview in **SDL** and bring up the **Design Migration** assistant to continue the following steps. Make sure migration settings, engines settings, and technology map all follow the same as previous tasks.



2. In the layout cellview, M1_CUT shapes are included and created at appropriate places. Also, a Track PCell instance is created to assist creating M2 vertical, M3 horizontal, and M4 vertical routes. Visually review the tracks with **Track Region Assistant** to enable the visibility of M2, M3 and M4 layers. Or query the Track PCell instance and click **Edit Tracks** in **Parameter** to see values of Width, Space, and other parameters. Close **Track Region Assistant** and the **Edit Tracks** dialog box.

Edit Tracks <@antlia>

Boundary LPP: border track

Pitch Width: 0.0738Pitch Height: 0.0448

☒ Visible☒ Active

☒ Show Routing Layers Only

M2

M3

M4

+

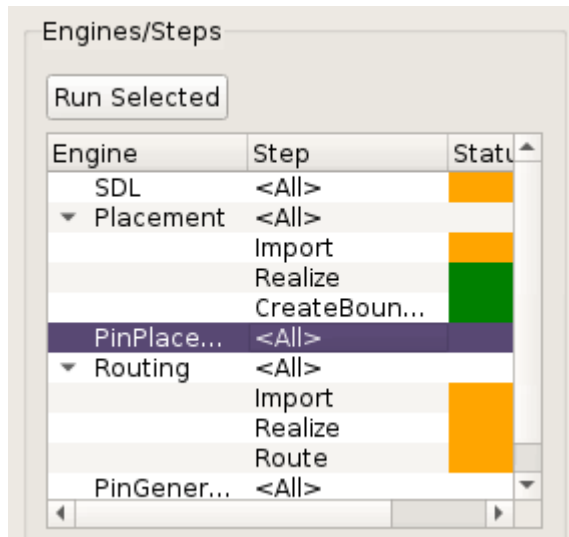
Group Attributes

Alias	Layer	Direction	Color Method	Draw Shape	Purpose	Uniform	Use Grids
	M2	vertical	auto	☐		☒	☐

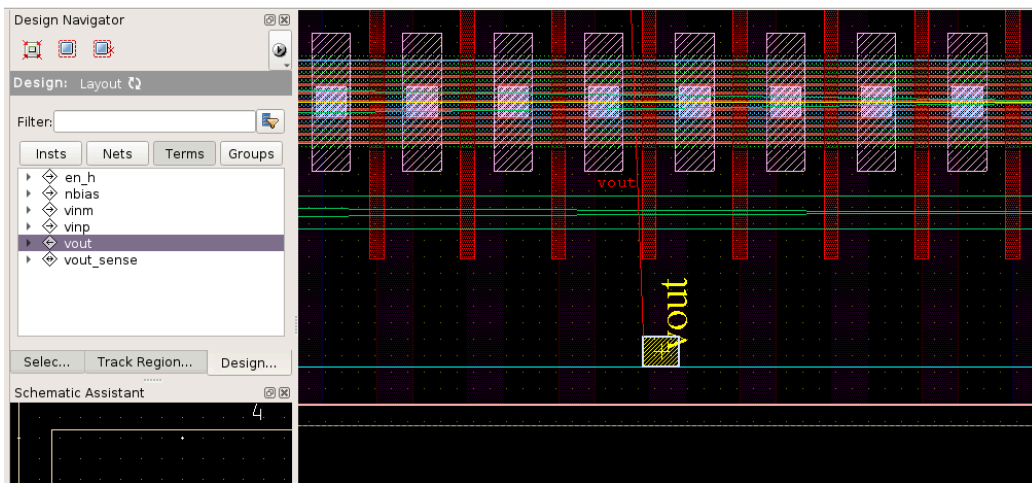
Tracks

	Alias	Offset	Width	Space	Type	Start Color	Color Shift	Low Grids	High Grids
1		0.053	0.0208	0.032	Non-Reserv...	red	☒		
	Click to Add								

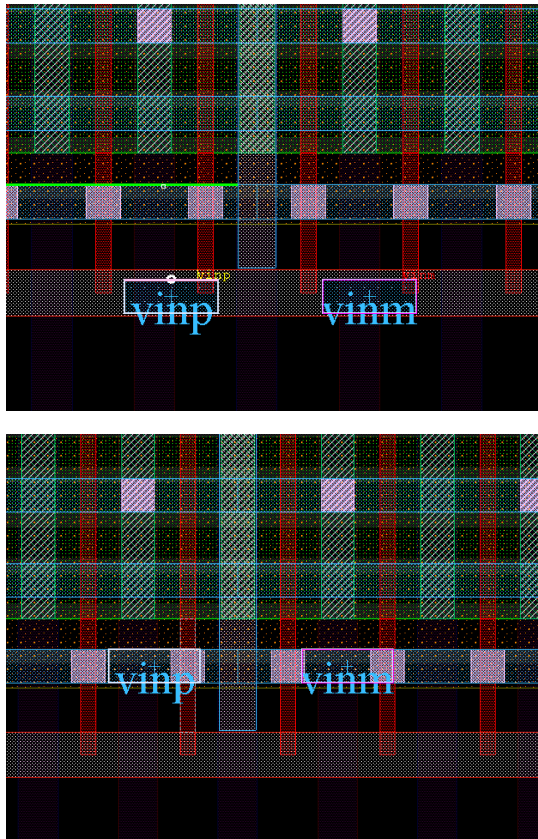
3. In **Design Migration** assistant, select **PinPlacement <All>** step and click **Run Selected** to migrate pin placement.



4. In **Design Navigator**, set **Design** to **Layout** and click the **Terms** tab to review the migrated boundary pin *vout*. Observe the pin layer *M2* and relative position replicated in the target design.



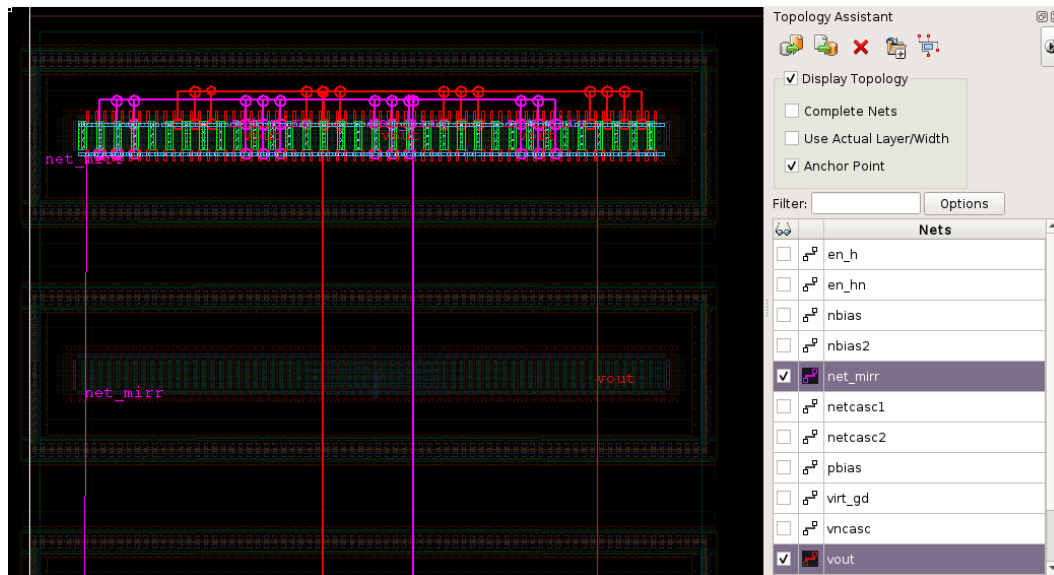
5. Review pin *vinp* and *vinm*, note these two M1 pins are not aligned perfectly with transistor gate pin metal track after migration. Select these two top-level pin shapes, invoke the Layout Editor command **align** with bindkey 4 to align the top edge of these two pins with the first metal track.



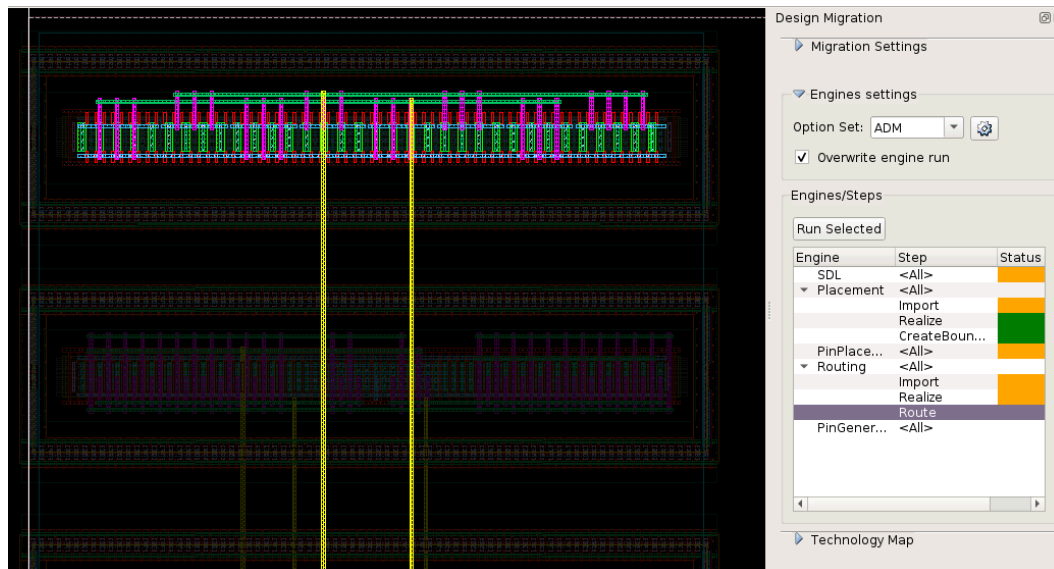
6. Finish reviewing Pin Placement result in layout canvas, switch back to **Design Migration** assistant.

Generating Target Design- Routing

1. Select the **Routing Import** step in the **Design Migration** assistant and click **Run Selected** to import routing topologies extracted from the source design.
2. Choose **Window > Assistants > Topology Assistant** to open the **Topology Assistant** pane. Enable **Display Topology** and click the  icon to disable visibility of all nets, and just enable the relevant nets.
3. Review the topologies extracted for the nets. Observe how the routing pattern and symmetry are retained. Toggle the **Use Actual Layer/Width** check box to display or hide with real metal layer width. Deselect **Display Topology** and close **Topology Assistant**.

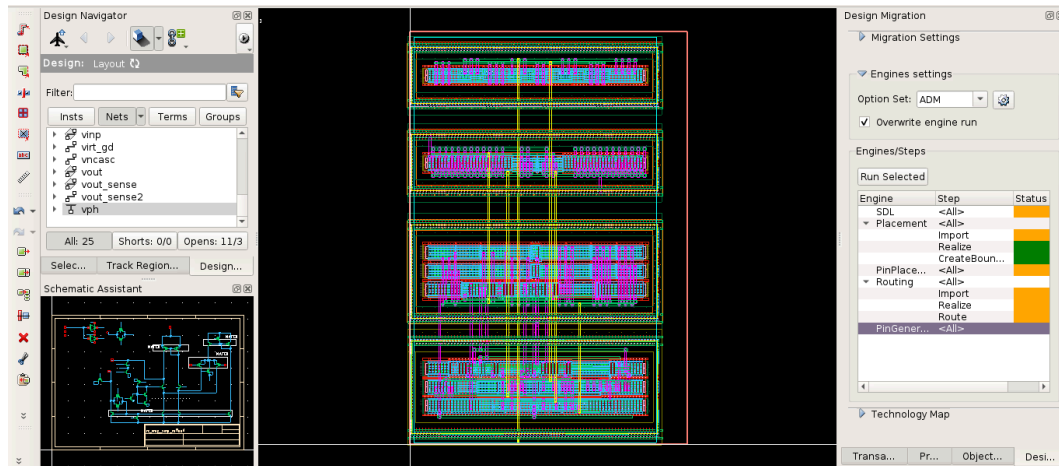


4. In **Design Migration** assistant, select the **Routing Realize** step and click **Run Selected** to realize the route topology. Observe in layout canvas how routing trunks are realized from topologies.



5. To route pins to trunk as well as the nets without topology extracted. Select the **Routing Route** step and click **Run Selected** to route remaining nets. Observe the routing created all follow the track defined in **Track PCell** instance. In **Design**

Navigator, click the **Nets** tab and notice that there are no short and the 3 open nets are Power/Ground nets.



Generating Target Design– Pin Generation

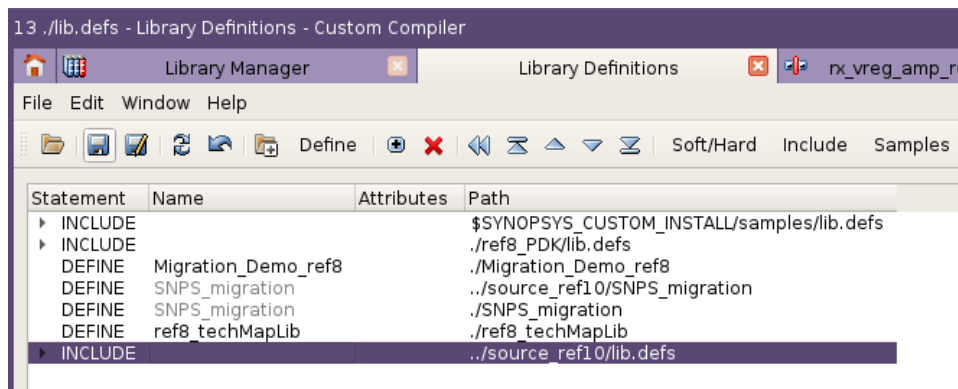
1. Finally, in **Design Migration** assistant, select the **PinGeneration <All>** step and click **Run Selected** to generate pin shares along the routing layers.
2. **Save Design** without closing it.

This concludes the **Design Migration** of a layout design.

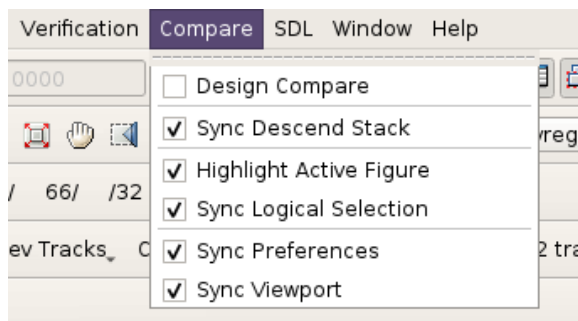
Comparing the Migrated Design with Source Design

To review and compare design with its original source to understand the similarities and differences using the **Design Compare** dialog box:

1. Choose **Library Manager > File > Library Definitions Editor** to include source design. Click **Include** and choose the source *lib.defs* file from file selector. Save and close the **Library Definition Editor** window.



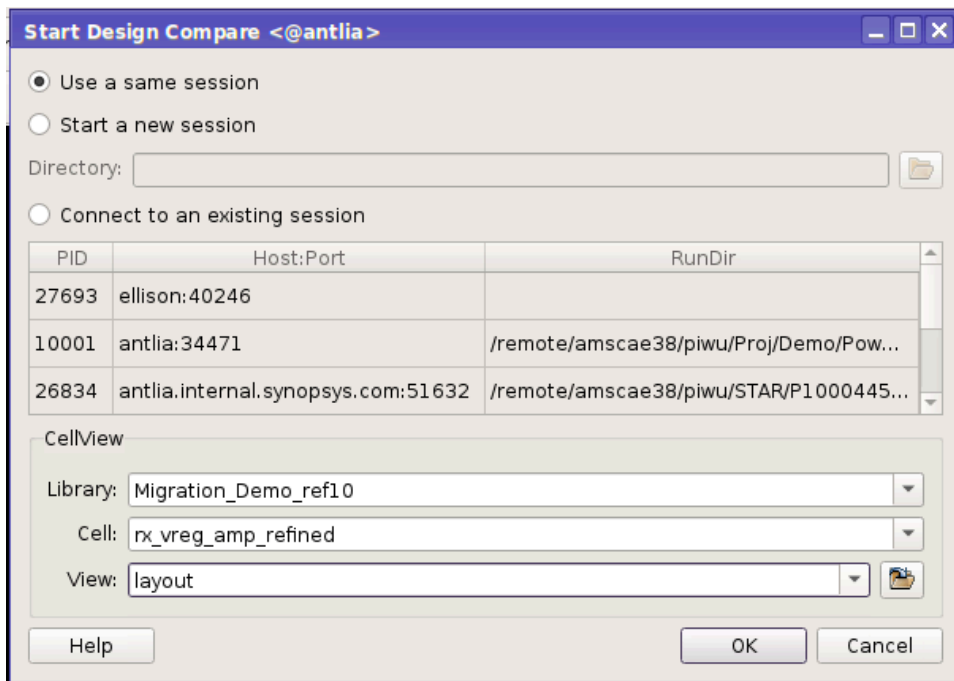
2. Switch back to Layout Editor with the design migrated in previous tasks. In the menu, choose **Tools > Design Compare** to enable the **Compare** menu.



Chapter 2: Layout Migration

Comparing the Migrated Design with Source Design

3. In the **Compare** menu, enable **Design Compare** to open the **Start Design Compare** dialog box. Select the **Use a same session** option, then set the source design layout cellview and click **OK**.



The compared designs are placed side by side and the viewport and logic selection changes are synchronized between these two designs. You can use this to select instances or nets to review placement and routing similarity of migrated design vs original source design.

Figure 9 Design review layout canvas

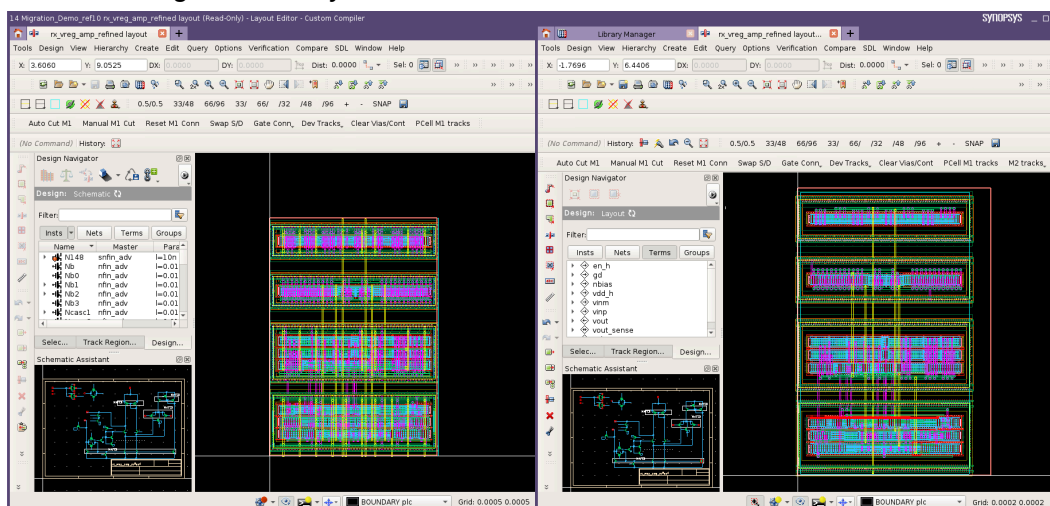


Figure 10 Review placement of Pb0 instance

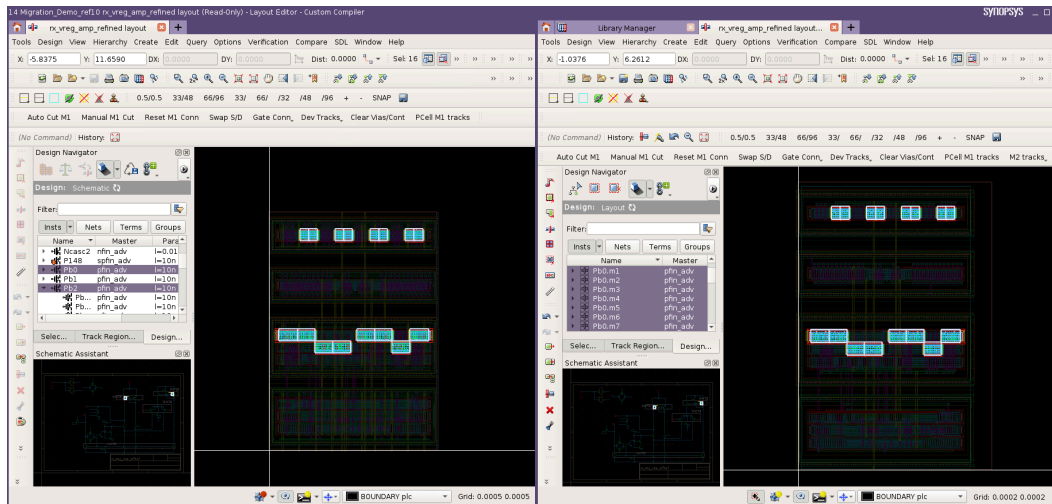


Figure 11 Review routing of vout net with Design Compare

